

Cloud Computing Architecture

Semester project report

Group 105

Author 1 - Legi-NR

Gianlorenzo Occhipinti - 21-981-170

Edouard Mignan - 21-806-807

Systems Group
Department of Computer Science
ETH Zurich
April 19, 2026

Instructions

- **Please do not modify the template!** Except for adding your solutions in the labeled places, and inputting your group number, names and legi-NR on the title page.

Divergence from the template can lead to subtraction of points on graded parts of the project.

- Parts 1 and 2 of the project are **ungraded**. They will help you analyze the behavior of the applications you will have to run on parts 3 and 4 (graded), for which you will have to design a scheduling policy **based on the information** you will gather on **parts 1 and 2**.
- Be conservative with your credits! Parts 3 and 4 might need extra credits for experimentation. Parts 1 and 2 should cost you approximately **15 USD**.
- **Use of AI Tools:** The use of AI tools must adhere to [ETH regulations](#). **You take responsibility for the content you submit.** You must disclose any use of GenAI and other AI tools in your work and avoid sharing copyrighted, private, or confidential information with commercial AI platforms. Failure to comply with these guidelines may result in disciplinary action.

Part 1

Using the instructions provided in the project description, run memcached alone (i.e., no interference), and with each iBench source of interference (cpu, l1d, l1i, l2, llc, membw). For Part 1, you must use the following `mcperf` command, which varies the target QPS from 5000 to 80000 in increments of 5000 (and has a warm-up time of 2 seconds with the addition of `-w 2`):

```
$ ./mcperf -s MEMCACHED_IP --loadonly
$ ./mcperf -s MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -w 2 -t 5 \
    --scan 5000:80000:5000
```

Repeat the run for each of the 7 configurations (without interference, and the 6 interference types) **at least 3 times** (3 should be sufficient in this case), and collect the performance measurements (i.e. the `client-measure` VM output).

Reminder: after you have collected all the measurements, make sure you **delete your cluster**. Otherwise, you will easily use up the cloud credits. See the project description for instructions how to make sure your cluster is deleted.

(a) Plot a line graph with the following stipulations:

- Queries per second (QPS) on the x-axis (the x-axis should range from 0 to 80K). (**note:** the actual achieved QPS, not the target QPS)
- 95th percentile latency on the y-axis (the y-axis should range from 0 to 6 ms).
- Label your axes.
- 7 lines, one for each configuration. Add a legend.
- State how many runs you averaged across and include error bars at each point in both dimensions.

Answer:

(b) How is the tail latency and saturation point (the “knee in the curve”) of memcached affected by each type of interference? What is your reasoning for these effects? Briefly describe your hypothesis.

Answer:

- `ibench-cpu`: Saturates early (22K QPS) with a very sharp elbow. p95 latency jumps from 0.6ms to 3.4ms at saturation.

Explanation: The CPU interferer takes away CPU time that memcached needs to process requests. Since there are fewer cycles available, fewer requests get handled per second, and the rest pile up waiting.

- `ibench-l1d`: Saturates at 33K QPS, p95 latency goes up to 3ms. Noticeable drop compared to the 45K with no interference.

Explanation: Memcached reads its data (hash table lookups, stored values) a lot. Polluting L1d means this data gets kicked out of the fastest cache, so the CPU has to fetch it from slower caches each time, making every request take longer.

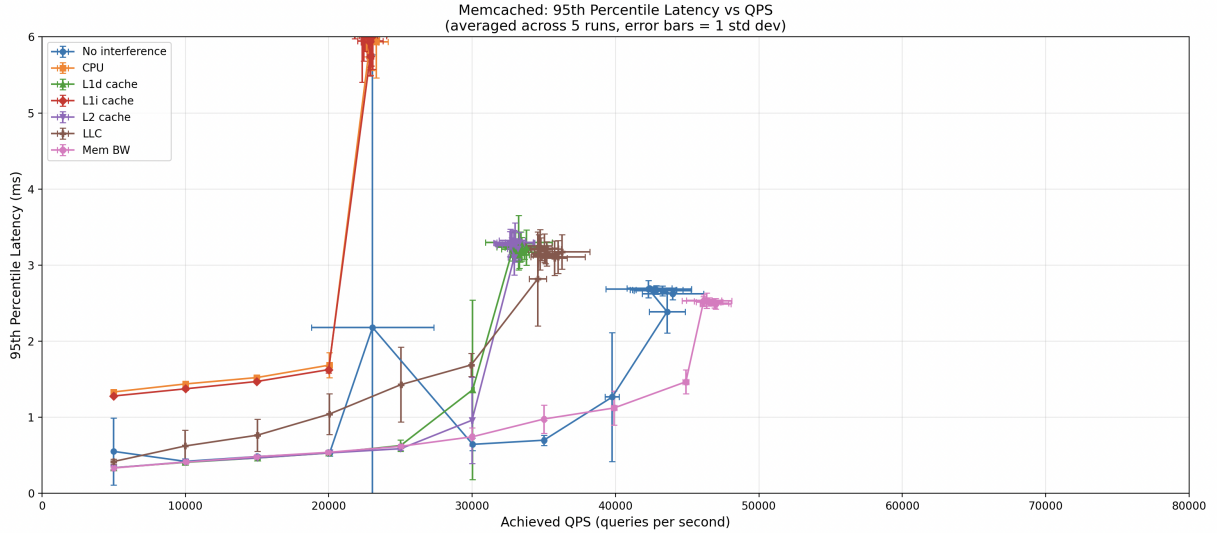


Figure 1: Memcached 95th percentile latency vs QPS

- **ibench-11i:** Saturates early at 22K QPS with p95 latency spiking to 6ms. One of the most damaging interferences.

Explanation: Even though memcached's hot loop is small, the L1i interferer runs on the same core and constantly evicts instructions from the cache. Since it shares core 0, it also competes for CPU cycles and other per-core resources, compounding the effect.

- **ibench-12:** Saturates around 33K QPS with p95 latency around 3ms. Similar impact to L1d.

Explanation: L2 is where L1 misses go. If L2 is also polluted, those misses have to go even further to LLC or main memory, which is much slower. So the penalty from L1d pollution gets worse since there's no fast fallback.

- **ibench-11c:** Saturates at 35K QPS, p95 latency around 2.9ms.

Explanation: LLC is the last cache before RAM. If it's polluted, any cache miss goes straight to main memory which is way slower. This adds up and reduces how many requests memcached can serve.

- **ibench-membw:** Saturates at 45K QPS with p95 latency staying around 1.2ms before saturation. Best performance among all interferences.

Explanation: This one floods the memory bus with traffic, so when memcached needs to read from RAM it has to wait for the bus to be free. It gets the best performance because it only slows down data transfer and doesn't steal CPU cycles or pollute caches.

(c) Processor affinity:

- Explain the use of the `taskset` command in the container commands for memcached and iBench in the provided scripts.

Answer: `taskset -c <core>` pins a process to a specific CPU core (CPU affinity), preventing the OS from moving it to another core. In our setup, memcached is pinned

to core 0. This is important because some experiments require the interferer to use the same core-private resources (like caches) as memcached, so we need to force both processes onto the same core.

- Why do we run some of the iBench benchmarks on the same core as memcached and others on a different core? Give an explanation for each iBench benchmark.

Answer:

- **ibench-cpu** (core 0): CPU cycles are per-core, so the interferer must run on the same core as memcached to compete for compute time.
- **ibench-l1d** (core 0): The L1 data cache is private to each core, so it must run on core 0 to evict memcached's data from that cache.
- **ibench-l1i** (core 0): The L1 instruction cache is also private per core, so same reasoning as L1d.
- **ibench-l2** (core 0): The L2 cache is private per core as well, so the interferer needs to share core 0 to pollute it.
- **ibench-llc** (core 1): The LLC is shared across all cores, so it can be polluted from any core. it hard to isolate the LLC effect.
- **ibench-membw** (core 1): The memory bus is shared across all cores, same idea as LLC.

- (d) Assuming a service level objective (SLO) for memcached of up to 2 ms 95th percentile latency at 35K QPS, which iBench source of interference can safely be colocated with memcached without violating this SLO? Briefly explain your reasoning.

Answer: Looking at the chart, the only iBench interference that can safely meet the SLO is **ibench-membw**. At 35K QPS, its p95 latency stays around 1.2ms, well below the 2ms threshold, and only exceeds it after 45K QPS. This is because the memory bandwidth interferer runs on a different core and only contends for the shared memory bus, which memcached is not heavily bottlenecked on at this load level. All other interferences either cannot reach 35K QPS or exceed the 2ms p95 latency at that throughput.

- (e) In the lectures you have seen queueing theory.

- Is the project experiment above (ignoring client measure) an open system or a closed system? Explain why.

Answer: It is an open system. The load generator (mcperv agent) sends requests at a fixed target rate specified by the `--scan` parameter, regardless of whether previous requests have been served. New arrivals are independent of the system state, which is the defining property of an open system.

- What is the number of clients in the system? How did you find this number?

Answer: There are 8 clients. This follows from the `-T 8` parameter used when launching the mcperv agent, which specifies 8 threads generating load against the memcached server.

- Sketch a diagram of the queueing system modeling the experiment above. Give a brief explanation of the sketch.

Answer: $\lambda \rightarrow \mu$ Requests arrive from the 8 agent threads at a combined rate λ (the target QPS). They enter a queue and are served by a single memcached thread (started with `-t 1`) at rate μ .

- Provide an expression for the average response time of this system. Explain each term in this expression and match it to the parameters of the project experiment.

Answer:

$$R = \frac{1}{\mu - \lambda}$$

Where:

- λ = arrival rate (achieved QPS), which increases from 5K to 80K in steps of 5K as set by `--scan 5000:80000:5000`
- μ = maximum service rate of memcached (requests/sec), i.e. the max throughput the server can sustain. This varies depending on the interference.
- R = average response time per request. As λ approaches μ , the denominator shrinks and R grows rapidly, which explains the latency spike (“elbow”) seen in the graph.

- (f) Below we ask you to do a theoretical cost analysis of the above experiments you ran and compare with the actual cost of your experiments, for which you will **need to track the total runtime** of the experiments, from deploying the cluster to its deletion. To find the actual cost, you can look at the **detailed billing report** after it is available on your Google Cloud project.

For the theoretical one, it would be helpful to check the official pricing calculator ([link](#)) or given individual pricing list ([link](#)). You also need to see the `machineType` and `subnets` parameters of each VM configuration by examining `part1.yaml`.

Note you can see the actual configuration details (e.g. disk size and type) of each VM configuration by looking at each individual VM instance after you deployed your cluster in `Compute Engine > VM instances > <your vm name>`.

- What is the cost per hour (in USD) for running each VM, according to the VM pricing?

Answer: **\$0.63**

- How much time (approximately) did your experiments take?

Answer: **2.5hrs**

- What is the expected (theoretical) total cost and what was the actual cost of running those experiments?

Answer: **\$1.63**

- Does the expected cost match the actual cost? (i.e. equivalent ± 0.01 if rounded up to 2 decimals). If they do not match, explain how you could achieve a more accurate estimate of the cost.

Answer: **yes!**

Costo totale (01–29 marzo 2026) ⓘ

Ultimi 7 giorni

Mese corrente

Costo

1,62 \$

—

Risparmi

1,62 \$

Mostra dettagli

=

Costo totale

0,00 \$

Costo totale previsto

Disponibile quando l'utilizzo
è sufficiente

\$1

Part 2

1. Interference behavior

- (a) Fill in the following table with the normalized execution time of each batch job with each source of interference. The execution time should be normalized to the job's execution time with no interference. Round the normalized execution time to 2 decimal places. Color-code each cell in the table as follows: **green** if the normalized execution time is less than or equal to 1.3, **orange** if the normalized execution time is over 1.3 and less or equal to 2, and **red** if the normalized execution time is greater than 2. The coloring of the first three cells is given as an example of how to use the cell coloring command. **Do not change the structure of the table. Only input the values inside the cells and color the cells properly.**

Workload	none	cpu	l1d	l1i	l2	l1c	memBW
barnes	1.00						
blackscholes	1.00						
canneal	1.00						
freqmine	1.00						
radix	1.00						
streamcluster	1.00						
vips	1.00						

Figure 2: Execution time of workloads with interference

- (b) Explain what the interference profile table tells you about the resource requirements for each job. Give your reasoning behind these resource requirements based on the functionality of each job.

Answer:

- **barnes:** Barnes is sensitive to CPU, L1 data and instruction cache, L2 cache and memory bandwidth interference (orange). It is very sensitive to last level cache interference (red).

Explanation: This profile is expected because the Barnes-Hut algorithm is an N-body simulation that involves heavy floating-point computation (CPU-bound) while traversing a hierarchical tree structure. Tree traversal causes irregular memory accesses that cause cache misses in lower level caches and rely heavily on the last level cache. It also demands significant memory bandwidth to fetch simulation data.

- **blackscholes:** Blackscholes is sensitive to CPU, L1 instruction cache, L2 cache, LLC, and memory bandwidth interference (orange) but not to L1 data cache interference (green).

Explanation: The blackscholes algorithm is used in finance for options pricing. It performs intensive mathematical (floating-point) operations, making it CPU-bound. It streams through large arrays of financial data sequentially, meaning data flows through the L1 data cache without being reused, so interference does not affect it.

However, it needs memory bandwidth for this streaming access and since it is CPU-bound it is also affected by L1 instruction cache interference.

- **canneal**: Canneal is not sensitive to CPU, L1 data or instruction cache interference (green) but is extremely sensitive to L2 and LLC interference (both red) and moderately to memory bandwidth interference (orange).

Explanation: Canneal is an optimization algorithm for minimizing the routing cost of a chip. It performs highly random accesses into a large chip netlist. The computation itself is light (green CPU), but relies heavily on the larger L2 and last level caches. The working data is too large to fit in those caches, making the job memory-latency-bound.

- **freqmine**: Freqmine shares the same profile as blackscholes (cpu, l1i, l2, llc, membw orange; l1d green).

Explanation: Frequent itemset mining involves scanning and processing transaction datasets in a largely sequential manner, making it compute and bandwidth intensive. Like blackscholes, it doesn't reuse data in the L1 data cache (hence green), but needs memory bandwidth and relies on the l2 and last level caches to reduce access times.

- **radix**: Radix is only sensitive to L2 and last level cache interference (orange), with everything else green.

Explanation: Radix sort is not particularly compute-heavy and accesses memory in a fairly structured way across multiple passes, which is friendly to L1 caches and doesn't saturate memory bandwidth. However, the intermediate sorting buffers and key/value arrays are large enough to create pressure on L2 and last level caches during the passes.

- **streamcluster**: Streamcluster has the same profile as radix (L2 and last level cache orange, all else green).

Explanation: As a streaming k-means clustering algorithm, it iterates over data points with reasonable locality, not relying much on the L1 data cache and not being very CPU-bounded. The algorithm will rely on L2 and last level caches to reduce access times which explains their sensitivity to interference.

- **vips**: Vips is identical to barnes (CPU, L1 data and instruction cache and Memory Bandwidth are orange; L2 cache is red).

Explanation: Image processing pipelines involve intensive pixel-level computation (CPU-bound) over large 2D data structures. This makes this job sensitive to interference all across the board. Operations like convolutions and transformations access data in patterns that stress all levels of cache and particularly the last level.

- (c) Which jobs (if any) seem like good candidates to colocate with memcached from Part 1, without violating the SLO of 2 ms P95 latency at 40K QPS? Explain why.

Answer: In part 1, memcached was most sensitive to CPU and L1 instruction cache interference, and also moderately sensitive to L1 data cache, L2 and last level cache interference. It was least sensitive to Memory Bandwidth interference. Given this infor-

mation, radix and streamcluster seem like good candidates to colocate with memcached since they are on the whole less resource-intensive and complementary to memcached.

2. Parallel behavior

- (a) Plot a line graph with speedup as the y-axis (normalized time to the single thread config, $\text{Time}_1 / \text{Time}_n$) vs. number of threads on the x-axis (1, 2, 4 and 8 threads - see the project description for more details). Pay attention to the readability of your graph.

Answer:

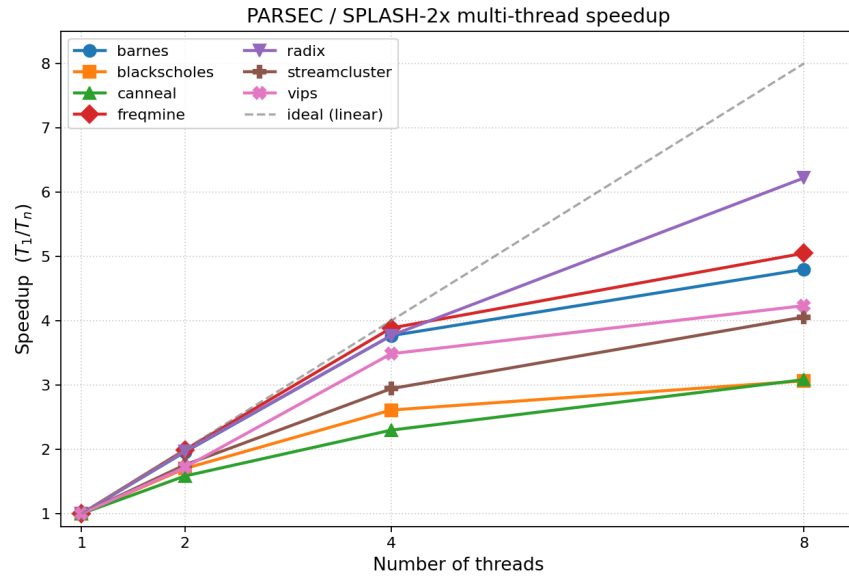


Figure 3: PARSEC multi-thread speedup

- (b) Briefly discuss the scalability of each job: e.g., linear/sub-linear/super-linear. Do the applications gain significant speedup with the increased number of threads? Explain what you consider to be “significant”.

Answer:

- barnes:
- blackscholes:
- canneal:
- freqmine:
- radix:
- streamcluster:
- vips:

Part 3 [68 points]

1. [34 points] Describe and analyze your hand-crafted policy.
 - (a) [17 points] With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time ¹ across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency > 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes	50.3	0.58
blackscholes	129.3	20.21
canneal	169.7	8.5
freqmine	102.7	4.16
radix	6.0	0.0
streamcluster	177.0	2.0
vips	147.0	10.39
total time	188.3	1.53

Table 1: Scheduling policy execution times

run	violations	rows	ratio
run1	0	15	0.0000
run2	0	15	0.0000
run3	0	15	0.0000
mean			0.0000
std			0.0000

Table 2: Memcached SLO violation ratio

¹Here, you should only consider the runtime, excluding time spans during which the container is paused.

Answer:

Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of mcpervf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. For each core in each machine show what job it is running using the colors proposed in this template (you can find them in `main.tex`). For example, draw a bar in **the color of vips** for core x from when vips is started to when it ends if vips ran on core x .

Plots:

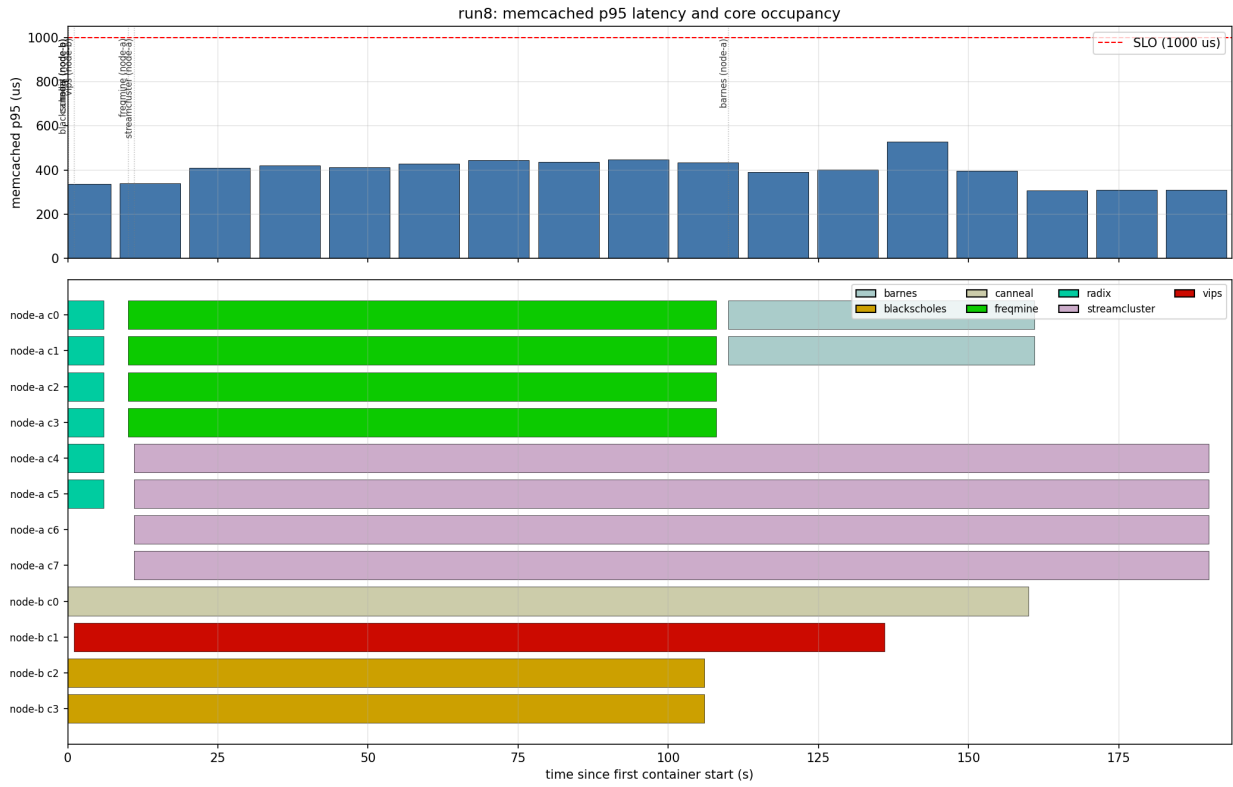


Figure 4: Evolution of memcached p95 latency across run 1 with workload node and core allocation.

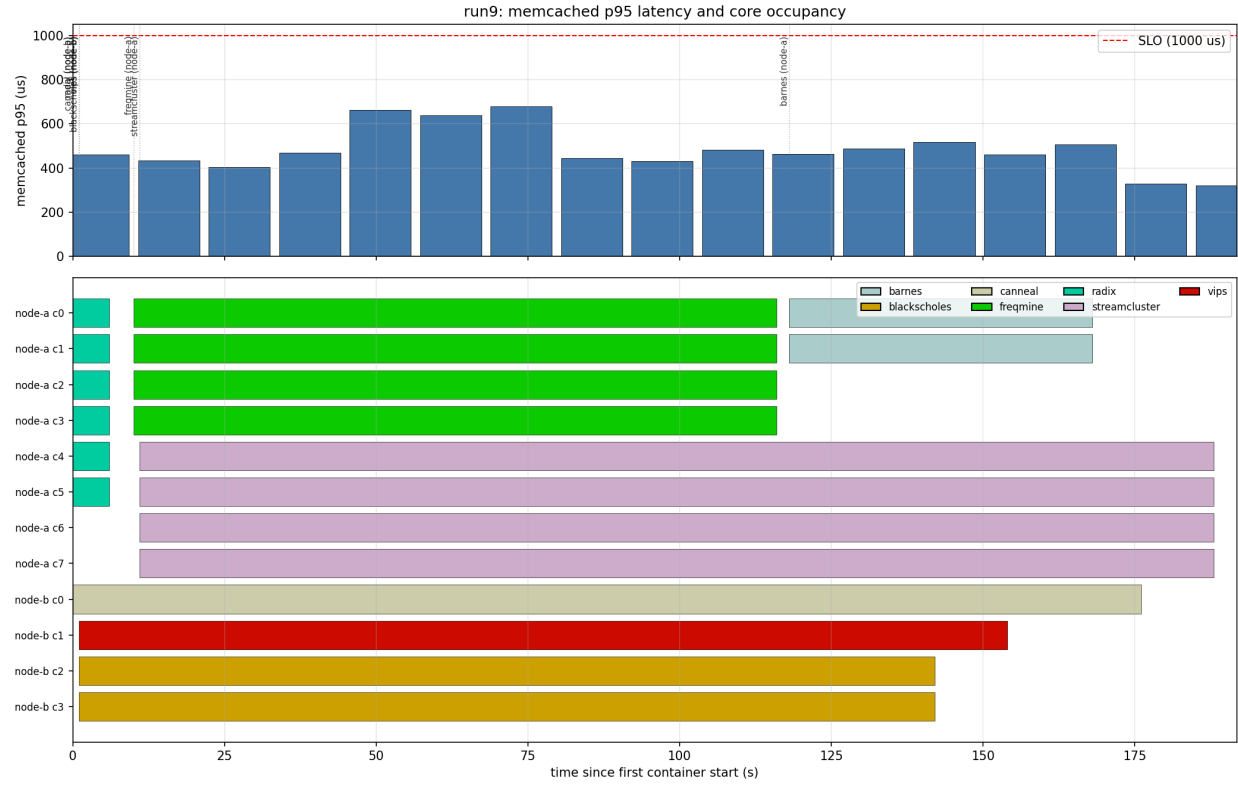


Figure 5: Evolution of memcached p95 latency across run 2 with workload node and core allocation.

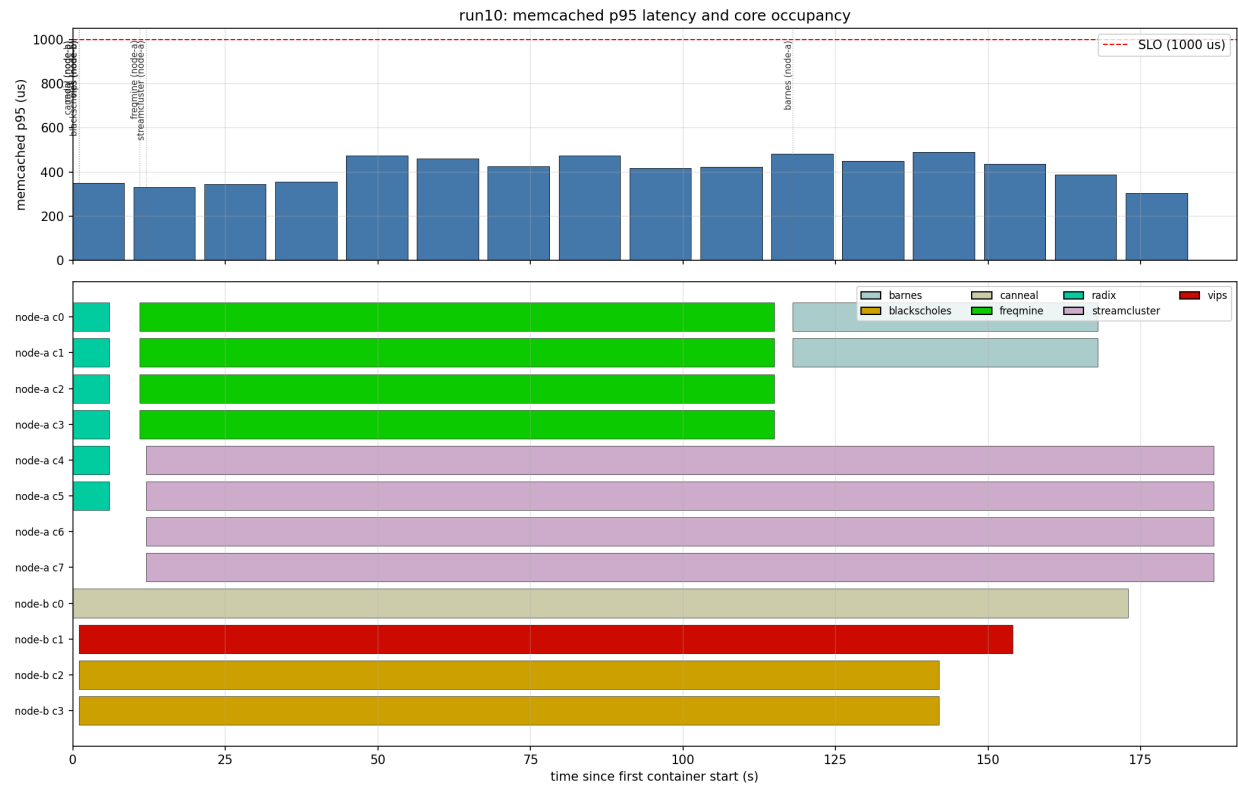


Figure 6: Evolution of memcached p95 latency across run 3 with workload node and core allocation.

- (b) [17 points] Describe and justify the “optimal” scheduling policy you have designed. To support our decision on where to place each job, we combined the data from Figure 1 at 30K QPS with Figure 2 into a single interference heatmap that shows how well each workload can co-exist with memcached under that load.

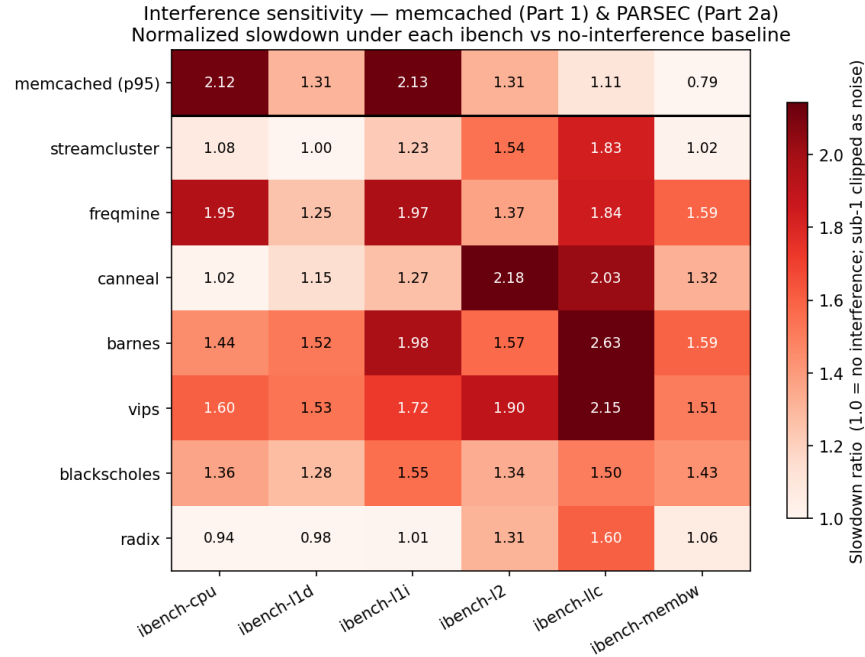


Figure 7: Interference heatmap of workloads against memcached at 30K QPS.

In addition, once we had chosen node-b as the home of memcached, we measured each workload running in isolation on both nodes to obtain a baseline execution time. The node-b runs were constrained to 2 cores, leaving the other 2 reserved as a conservative estimate for memcached’s share.

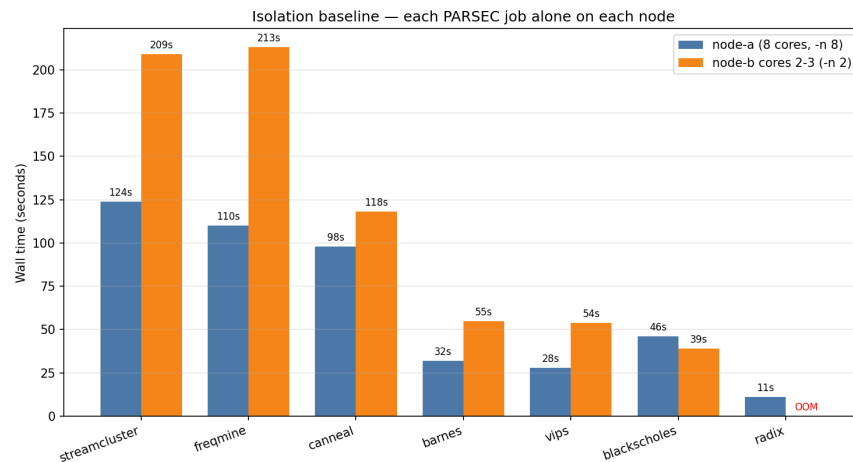


Figure 8: Baseline execution time of each job on node-a and node-b in isolation.

- Which node does memcached run on? Why?

Answer: Memcached runs on **node-b**. We split the workloads so that the CPU-intensive jobs go on node-a (8 cores), while memcached and the jobs that interfere with it the least share node-b. Part 1 showed that memcached can sustain 30K QPS with only two cores, so the smaller node is not a limit for the service.

- Which node does each of the 7 batch jobs run on? Why?

Answer:

- **barnes**: runs on **node-a**. Our sample execution times show it runs about twice as fast on node-a as on node-b, and it generates significant L1i pressure, to which memcached is sensitive, making it a poor neighbour for memcached.
- **blackscholes**: runs on **node-b**. Even with only 2 cores it had a better runtime than on node-a (Fig. 8). It puts some CPU and L1i pressure on memcached, but Figure 3 shows that it does not benefit much from a multi-thread setup, which makes it an ideal candidate for node-b.
- **canneal**: runs on **node-b**. From Figure 7 we see it is a moderate but tolerable interferer for memcached, and Figure 3 confirms it does not gain much from multi-threading, which makes it an ideal candidate for node-b.
- **freqmine**: runs on **node-a**. The interference heatmap in Figure 7 shows it is not a good roommate for memcached, and Figure 3 shows that it scales well with the number of threads, which makes it an ideal candidate for a multi-threaded setup on node-a.
- **radix**: runs on **node-a**. The choice is forced: the memory request of this workload rules out node-b on memory alone. It is also the workload that benefits most from multi-threading, as shown in Figure 3.
- **streamcluster**: runs on **node-a**. From Figure 8 it clearly benefits from the extra resources on node-a, with a speedup of almost 41% compared to node-b.
- **vips**: the ideal node for vips would be node-a, but since node-a is already loaded we placed it on **node-b** to keep the workload balanced and reduce the overall wall-time.

- Which jobs run concurrently / are colocated? Why?

Answer: There are two groups of jobs that run concurrently:

- On **node-b**: **canneal**, **blackscholes** and **vips** run together with memcached for the whole duration of the run. We tuned the YAML files to make their combined CPU requests fit under node-b's allocatable cores, so the Kubernetes scheduler admits all four pods together. Figure 4 confirms that none of the three batch jobs pushes memcached's p95 over the 1 ms SLO at 30K QPS.
- On **node-a**: **freqmine** and **streamcluster** are co-scheduled in phase 1. From Figure 8 and Figure 3 we saw that both scale at roughly 45% parallel efficiency, so we picked a balanced CPU request split of 3500m for freqmine and 4 for streamcluster. The two requests fit under node-a's 8 cores, and the balanced split makes both jobs finish at roughly the same time.

radix and **barnes** are *not* co-located with any other batch job on node-a: radix runs alone in phase 0 because it is a strong memory-bandwidth interferer, and barnes runs alone in phase 2 once one of the big jobs has finished and freed the cores.

- In which order did you run the 7 batch jobs? Why?

Order: **canneal**, **blackscholes**, **vips**, **radix**, **freqmine**, **streamcluster**, **barnes**

Why: We submit first the batch of node-b jobs and then the batch of node-a (doing the opposite won't change the result, this is an arbitrary choice). `radix` is then run alone on node-a in phase 0: it is a strong memory-bandwidth interferer and co-locating it with anything else on node-a would hurt both jobs, but running it alone with 8 threads makes it finish in about 6 seconds. Once `radix` is complete, we submit `freqmine` and `streamcluster` one second apart; the 1-second gap guarantees that `freqmine`'s `creationTimestamp` precedes `streamcluster`'s, which lets the Kubernetes scheduler admit the balanced pair in the right order. `barnes` is submitted last on purpose: its small 2-core request would otherwise be admitted before the big pair and could block them, so by submitting it last it stays Pending until one of `freqmine`/`streamcluster` finishes and is admitted naturally in phase 2.

- How many threads have you used for each of the 7 batch jobs? Why?

Answer:

The CPU limits of each job (and therefore the thread counts) were chosen by observing workload behaviour and node load in real time, using the `htop` command on both nodes while jobs were being scheduled. A more rigorous approach would have been to install the Kubernetes Metrics API server, which would have given us a clearer profile of each workload's actual CPU and memory usage instead of the informal picture we obtained through `htop`.

- `barnes`: **4 threads**. Matches its CPU limit of 4 cores, which is what `barnes` can actually use in phase 2 when it runs alone on node-a. Figure 3 shows diminishing returns past 4 threads.
 - `blackscholes`: **2 threads**. Matches the `taskset -c 2-3` pin on node-b, which gives it exactly 2 physical cores. Using more threads would only add context switches on the same two cores.
 - `canneal`: **4 threads**. Unpinned on node-b, so 4 threads match the 4 physical cores of node-b. Keeping all 4 threads active lets `canneal` pick up idle cycles whenever `memcached`, `blackscholes` or `vips` are quiet.
 - `freqmine`: **8 threads**. Unpinned on node-a, so 8 threads cover all 8 cores. Its CFS share under contention with `streamcluster` is 3500m, but keeping 8 threads alive means every core can run a `freqmine` thread whenever `streamcluster` is scheduled out.
 - `radix`: **8 threads**. Runs alone on node-a in phase 0, and Figure 3 shows `radix` has the best multi-thread scaling of the seven workloads, so using all 8 cores is the fastest possible setup.
 - `streamcluster`: **8 threads**. Same reasoning as `freqmine`: unpinned on node-a, 8 threads match the 8 cores, and its CFS share of 4 cores is kept busy at all times by letting threads fill whichever core is free.
 - `vips`: **4 threads**. Unpinned on node-b, so 4 threads match the 4 cores of node-b. Its CPU limit of 3 caps the aggregate CPU it can consume, but keeping 4 threads lets it use cores 0-1 whenever `memcached` is idle.
- Which files did you modify or add and in what way? Which Kubernetes features did you use?

Answer:

We modified every workload YAML to declare explicit CPU and memory **requests** and **limits**, letting the Kubernetes scheduler decide the timing on when pods were

scheduled. Specifically:

- **PARSEC YAMLs:** each job sets a `nodeSelector` to pin it to node-a or node-b, declares CPU/memory requests and limits, and tunes the `-n N` flag so the thread count matches the number of cores we expect it to receive. For the two jobs `parsec-canneal`, `parsec-blackscholes`, we used `taskset -c` inside the container command to pin them to a specific CPU range.
- **memcached YAML:** we removed the original `taskset` pinning, set `-t 4` so the server uses all four cores of node-b, and added `requests` of 1500m CPU / 512Mi memory with `limits` of 4 CPU / 1Gi.

The Kubernetes features we relied on are:

- `nodeSelector` on every pod to force placement onto node-a or node-b.
- Resource `requests` and `limits`, so the scheduler only admits a pod when the target node has enough free capacity.
- The goal was to have a QoS class *Burstable* (request < limit) for every pod, which lets batch jobs use idle cores without compromising memcached’s guaranteed resources.

In addition, we added a set of orchestration and analysis scripts to streamline our experiment setup.

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).

Answer: Our main constraint was to never violate the 30K QPS SLO given by the assignment, so we closely observed how memcached reacted while tuning resource allocations and scheduling jobs. In our best policy the p95 latency settled around 500 μ s, well below the 1 ms target, which suggests some further optimisation might still be possible. Hitting a wall time of 188 s, given that the slowest execution time of a job on a dedicated node is 124 s, makes us feel we went in the right direction. We managed to squeeze the other six jobs into less than 60 s with parallelization, which is surely a meaningful result.

Our second-best policy reached a wall time of 203 s (single run) and differed from the best one by scheduling vips on node-a instead of node-b. By monitoring live node usage with `htop` while jobs were running, we noticed that node-b went idle earlier than node-a; this was the rationale behind moving vips to node-b, rebalancing the load between the two nodes.

A future direction for this project would be to close the loop between measurement and scheduling. Concretely, we would first try to deploy the Kubernetes `metrics-server` and collect live CPU and memory usage per job across a few trial runs, to then use that data to refine each job’s `requests` and `limits` (instead of relying on manual `htop` observations), and second, try to encode the interference profiles measured in Part 2 as `podAntiAffinity` rules, so that the scheduler itself keeps memcached away from the PARSEC jobs that stress the caches and CPU most heavily. Both directions build on our current policy rather than replacing it, and would reduce the amount of manual tuning needed to reach a similar result.

2. [34 points] Describe and analyze the AI-generated policy. [TODO: Xiaozhe]

(a) [17 points] **Report the performance:**

- Report the mean time of each batch job, as well as the makespan of all jobs for the AI-generated policy. Also, report the SLO violation ratio for memcached for the AI-generated policy. *Note: If the LLM failed to produce a valid solution after 3+ attempts, provide the error logs and your best-performing manual tweak for partial credit.*

job name	mean time [s] (Baseline)	mean time [s] (AI)
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

- Create 3 bar plots (one for each run), for the final AI-generated policy, of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Use the same method as described in the previous question to create the bar plots.

Answer:

- Create two line plots (one for the p95 latency and one for SLO violations) showing policy performance over iterations.

Answer:

(b) [17 points] **Analysis of the Evolutionary Process:** Describe the process of how you used OpenEvolve to design the AI-generated policy. For example, you can include the following details in your answer (You are not limited to these questions. You can include any other details you find relevant and interesting). Your response will be graded based on the thoroughness of the exploration and clarity in explanation:

- How many iterations did you run through OpenEvolve with the LLMs to get to the final policy? Which LLM(s) did you use?
- How do you measure success? For example, how do you design the ‘combined_score’ or the success metric, given that there are two objectives (makespan and SLO violations)?
- Describe the initial policy you fed to the LLM(s) and why you chose this as the initial policy.
- Describe how the policy changes during the process, and how it impacted the performance of the policy (e.g., tail latency; SLO violations).
- Explain why did this AI-generated policy perform better (or worse) than the baseline?
- Identify one “hallucination” or logical flaw the model produced during the process and how you identified it.

Answer:

Please attach your modified/added YAML files, run scripts, OpenEvolve scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to design a policy that has a reasonable running time, does not violate the SLO, and takes into account the characteristics of the first two parts of the project.

Part 4 [75 points]

1. [19 points] Use the following `mcperf` command to vary QPS from 5K to 125K in order to answer the following questions:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 2 \
    --scan 5000:125000:10000
```

a) [8 points] How does memcached performance vary with the number of threads (T) and number of cores (C) allocated to the job? In a single graph, plot the 95th percentile latency (y-axis) vs. achieved QPS (x-axis) of memcached (running alone, with no other jobs collocated on the server) for the following configurations (one line each):

- Memcached with $T=1$ thread, $C=1$ core
- Memcached with $T=1$ thread, $C=2$ cores
- Memcached with $T=1$ thread, $C=3$ cores
- Memcached with $T=2$ threads, $C=1$ core
- Memcached with $T=2$ threads, $C=2$ cores
- Memcached with $T=2$ threads, $C=3$ cores
- Memcached with $T=3$ thread, $C=1$ cores
- Memcached with $T=3$ thread, $C=2$ cores
- Memcached with $T=3$ threads, $C=3$ cores

Label the axes in your plot. State how many runs you averaged across (we recommend three runs) and include error bars. The readability of your plot will be part of your grade.

Plots:

What do you conclude from the results in your plot? Summarize in 2-3 brief sentences how memcached performance varies with the number of threads and cores and how this relates to the target QPS.

Summary:

b) [2 points] To support the highest load in the trace (around 125K QPS) without violating the 0.8ms latency SLO, how many memcached threads (T) and CPU cores (C) will you need?

Answer:

c) [1 point] Assume you can change the number of cores allocated to memcached dynamically as the QPS varies from 5K to 125K, but the number of threads is fixed when you launch the memcached job. How many memcached threads (T) do you propose to use to guarantee the 0.8ms 95th percentile latency SLO while the load varies between 5K to 220K QPS?

Answer:

d) [8 points] Run memcached with the number of threads T that you proposed in (c) and measure performance with $C = 1$, $C = 2$ and $C = 3$. Use the aforementioned `mcperf` command to sweep QPS from 5K to 125K.

Measure the CPU utilization on the memcached server at each load time step. Plot the performance of memcached using 1-core ($C = 1$), 2 cores ($C = 2$) and 3 ($C = 3$) in **three separate graphs**, for $C = 1$, $C = 2$ and $C = 3$, respectively. In each graph, plot achieved QPS on the x-axis, ranging from 0 to 125K. In each graph, use two y-axes. Plot the 95th percentile latency on the left y-axis. Draw a dotted horizontal line at the 0.8ms latency SLO. Plot the CPU utilization (ranging from 0% to 100% for $C = 1$, 200% for $C = 2$ and 300% for $C = 3$) on the right y-axis. For simplicity, we do not require error bars for these plots.

Plots:

2. [17 points] You are now given a dynamic load trace for memcached, which varies QPS randomly between 5K and 110K in 15 second time intervals. Use the following command to run this trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
    --qps_interval 15 --qps_min 5000 --qps_max 110000
```

Note that you can also specify a random seed in this command using the `--qps_seed` flag.

For this and the next questions, feel free to reduce the `mcperf` measurement duration (`-t` parameter, now fixed to 30 minutes) as long as you have at the end at least 1 minute of memcached running alone.

Design and implement a controller to schedule memcached and the benchmarks (batch jobs) on the 4-core VM. The goal of your scheduling policy is to successfully complete all batch jobs as soon as possible without violating the 0.8ms 95th percentile latency for memcached. **Your controller should not assume prior knowledge of the dynamic load trace. You should design your policy to work well regardless of the random seed.** The batch jobs need to use the native dataset, i.e., provide the option `-i native` when running them. Also make sure to check that all the batch jobs complete successfully and do not crash. Note that batch jobs may fail if given insufficient resources.

Describe how you designed and implemented your scheduling policy. Include the source code of your controller in the zip file you submit.

- Brief overview of the scheduling policy (max 10 lines):

Answer:

- How do you decide how many cores to dynamically assign to memcached? Why?

Answer:

- How do you decide how many cores to assign each batch job? Why?

Answer:

- `barnes`:
- `blackscholes`:
- `canneal`:
- `freqmine`:
- `radix`:
- `streamcluster`:
- `vips`:

- How many threads do you use for each of the batch job? Why?

Answer:

- `barnes`:
- `blackscholes`:
- `canneal`:
- `freqmine`:
- `radix`:
- `streamcluster`:
- `vips`:

- Which jobs run concurrently / are colocated and on which cores? Why?

Answer:

- In which order did you run the batch jobs? Why?

Order (to be sorted): `barnes`, `blackscholes`, `canneal`, `freqmine`, `radix`, `streamcluster`, `vips`

Why:

- How does your policy differ from the policy in Part 3? Why?

Answer:

- How did you implement your policy? e.g., docker cpu-set updates, taskset updates for memcached, pausing/unpausing containers, etc.

Answer:

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).

Answer:

3. [23 points] Run the following `mcperrf` memcached dynamic load trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
    --qps_interval 15 --qps_min 5000 --qps_max 110000 \
    --qps_seed 2345
```

Measure memcached and batch job performance when using your scheduling policy to launch workloads and dynamically adjust container resource allocations. Run this workflow 3 separate times. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached. For each batch application, compute the mean and standard deviation of the execution time² across three runs. Compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Also, compute the SLO violation ratio for memcached for each of the three runs; the number of data points with 95th percentile latency > 0.8ms, as a fraction of the total number of datapoints. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

Answer:

Include three plots – one for each of the three runs – with the following information: The x-axis should be the time from execution start to end. Use three subplots that share the same x-axis. The first should clearly indicate for each core what job it is running at all time (e.g. using colored boxes). Use the colors proposed in this template (you can find them in `main.text`). If you pause/unpause jobs as part of your scheduling policy this should also be clearly visible. The second subplot should include two y-axis where the left one shows the current QPS and the right one should include the achieved p95 latency. Finally, the third subplot should plot the cpu utilization of each core separately. The readability of your plot will be part of awarded points.

Plots:

4. [16 points] Repeat Part 4 Question 3 with a modified mcperf dynamic load trace with a 5 second time interval (`qps_interval`) instead of 10 second time interval. Use the following command:

²Here, you should only consider the runtime, excluding time spans during which the container is paused.


```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
    --qps_interval 5 --qps_min 5000 --qps_max 110000 \
    --qps_seed 2345
```

You do not need to include the plots or table from Question 3 for the 5-second interval. Instead, summarize in 2-3 sentences how your policy performs with the smaller time interval (i.e., higher load variability) compared to the original load trace in Question 3.

Summary:

What is the SLO violation ratio for memcached (i.e., the number of datapoints with 95th percentile latency > 0.8ms, as a fraction of the total number of datapoints) with the 5-second time interval trace? The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

Answer:

What is the smallest `qps_interval` you can use in the load trace that allows your controller to respond fast enough to keep the memcached SLO violation ratio under 3%?

Answer:

What is the reasoning behind this specific value? Explain which features of your controller affect the smallest `qps_interval` interval you proposed.

Answer:

Use this `qps_interval` in the command above and collect results for three runs and fill out the table below.

job name	mean time [s]	std [s]
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

Plots:

Use of AI Tools: Did you use any AI tools for this project? If so, disclose them here and explain what you used the tool(s) for and why. Note that the use of AI tools is NOT needed and NOT expected for the project.

Answer: We used AI assistance only for auxiliary tasks that did not affect the design of our scheduling policy. Specifically, we relied on it to speed up the creation of the Python plotting scripts

used to generate the charts in this report, and to draft the orchestration scripts that streamline experiment execution, collecting per-run results, parsing **mcperf** output, aggregating metrics across runs, and producing summary tables. All scheduling decisions, resource-request and limit values, workload placement, and the analysis of the results are our own.